

CCA-F

Claude Certified Architect — Foundations

Exam-Ready Study Kit

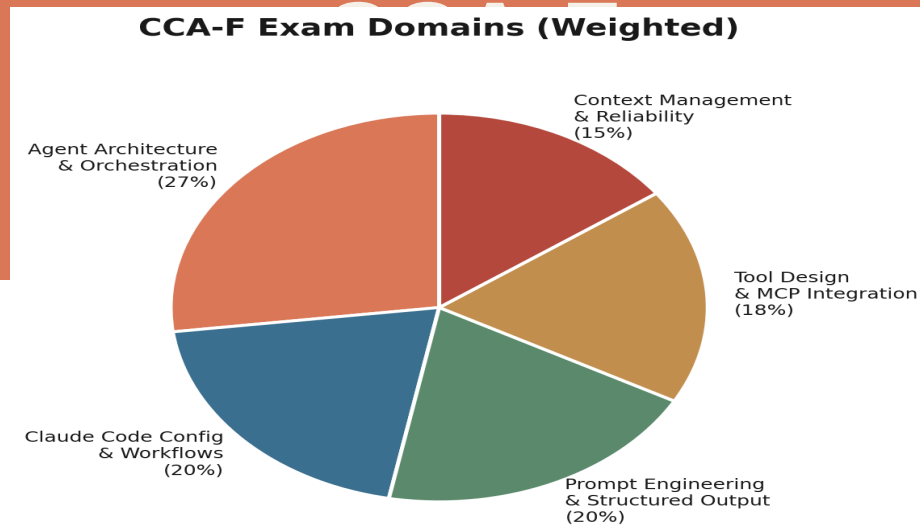
Claude Certified Architect — Foundations

Duration: 120 minutes · 60 questions

Format: Multiple choice (1 correct of 4) — no guessing penalty

Scoring: 100 – 1000 scale; passing = 720

Delivery: Online proctored via Anthropic Skilljar



Built for the May 2026 exam blueprint. Covers the Claude API, Claude Agent SDK, Model Context Protocol (MCP), Claude Code, prompt engineering, and reliability patterns.

Contents

CCA-F

Claude Certified Architect — Foundations

1. Exam Overview & Roadmap
2. Theory — Chapters 1–13
3. Domain-by-Domain Notes
4. Exam Scenarios (8)
5. Quick-Reference Cheat Sheet
6. Practice Questions with Explanations
7. Practical Exercises (Hands-on Labs)
8. Study Roadmap & Recommended Resources
9. Out-of-Scope Topics
10. Glossary

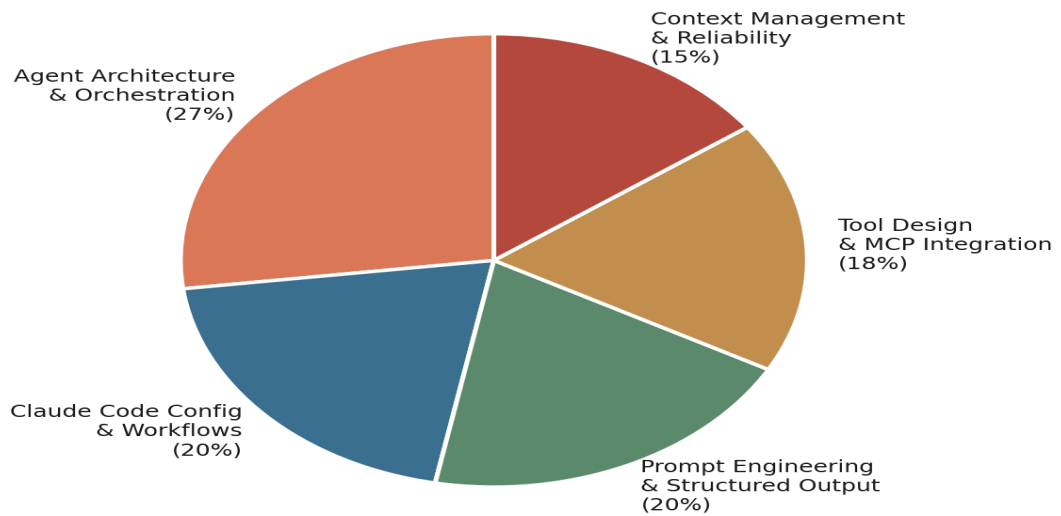
1. Exam Overview

The Claude Certified Architect — Foundations (CCA-F) certification validates your ability to make sound architectural trade-offs when shipping production applications with Claude. The exam draws on five weighted domains and is delivered as scenario-driven multiple-choice questions.

Parameter	Value
Question type	Multiple choice (1 correct of 4) — no guessing penalty
Scenarios sampled	4 of 8 scenarios are sampled per attempt
Number of questions	60
Duration	120 minutes
Scoring	100 – 1000 scale; passing = 720
Delivery	Online proctored via Anthropic Skilljar

Domain Weighting

CCA-F Exam Domains (Weighted)



Domain	Weight
1. Agent Architecture & Orchestration	27%
2. Claude Code Configuration & Workflows	20%
3. Prompt Engineering & Structured Output	20%

4. Tool Design & MCP Integration	18%
5. Context Management & Reliability	15%

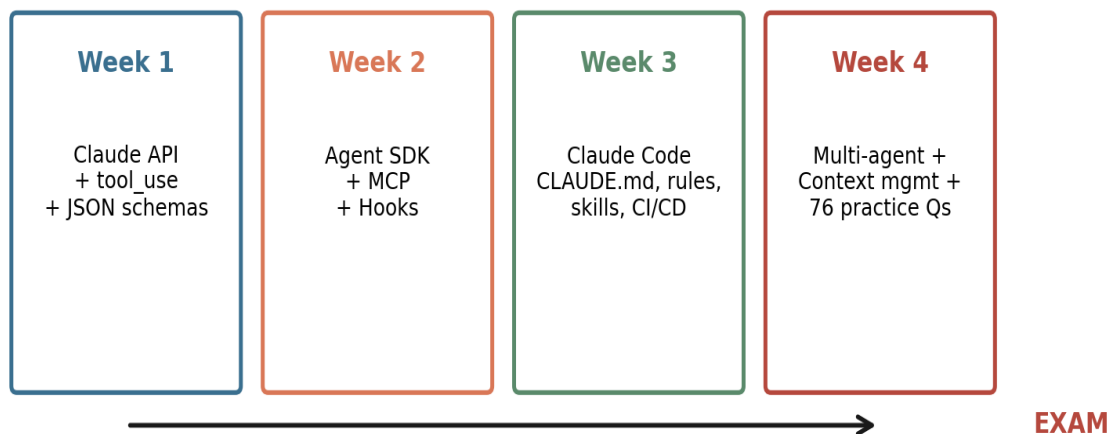
The 8 Production Scenarios

Every attempt randomly samples 4 of these 8 scenarios. Questions are rooted in realistic system-design situations rather than rote facts.

- Customer Support Agent
- Code Generation with Claude Code
- Multi-agent Research System
- Developer Productivity Tools
- Claude Code for Continuous Integration
- Structured Data Extraction
- Conversational AI Architecture Patterns
- Agentic AI Tools

Suggested 4-Week Study Plan

Suggested 4-Week Study Roadmap



Official Skilljar Preparation Tracks (free)

- AI Fluency: Framework & Foundations
- Introduction to Agent Skills
- Building with the Claude API
- Claude Code in Action
- Intro to Model Context Protocol
- MCP: Advanced Topics
- Prompt Engineering Interactive Tutorial
- Real-world Prompting
- Tool Use with Claude
- Customer Service Agent (cookbook walkthrough)
- Multi-agent Research Pipelines
- Structured Data Extraction Workshop
- Claude Code in CI/CD

2. Theory — The 13 Chapters

These chapters distil what every CCA-F candidate must internalise. They are organised by technology and concept (not exam domain) so the underlying mental model is built before mapping back to the blueprint.

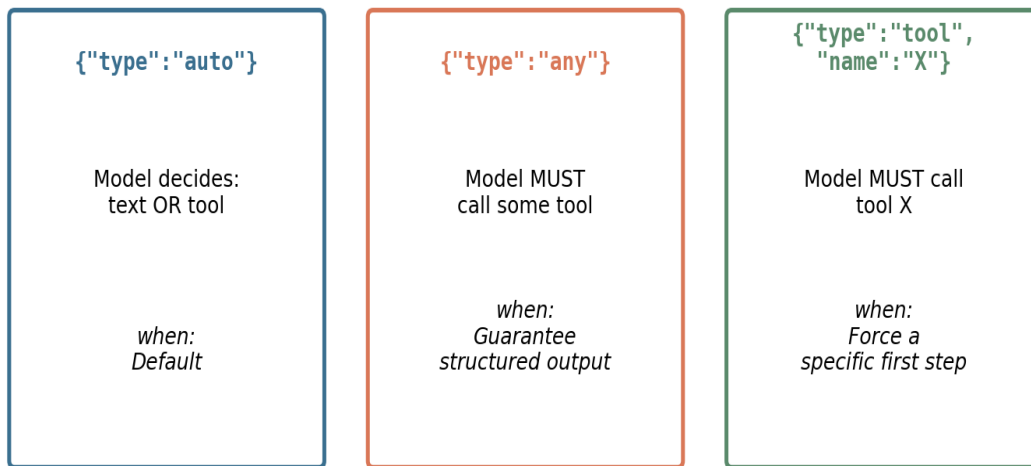
1. Claude API — Fundamentals

- Stateless: every request must include the FULL prior conversation in ``messages``. The model has no server-side memory.
- Request fields: ``model``, ``max_tokens``, ``system``, ``messages``, ``tools``, ``tool_choice``.
- Roles: ``user``, ``assistant``, plus tool results (sent as a ``tool_result`` content block on a user-role message).
- `stop_reason` values you must handle: ``end_turn`` (final answer), ``tool_use`` (run the tool, append result, loop), ``max_tokens`` (truncated — increase limit), ``stop_sequence``.
- System prompt: separate field; defines persistent role/constraints/format. Wording can create unintended tool associations (e.g. forcing overuse of `get_customer`).
- Context window includes system prompt + full history + tool definitions + tool results. Watch for: (a) lost-in-the-middle effect, (b) tool-result accumulation, (c) detail loss in progressive summarisation.

2. Tools and `tool_use`

- `tool_use` lets Claude REQUEST a tool call (a structured JSON block); your code runs the function and feeds the result back.
- Tool description is the PRIMARY selection signal. Minimal descriptions ('Gets customer info') cause tool confusion.
- Each description should include: what it does/returns, input formats with examples, edge cases, when to use vs similar alternatives.
- Avoid overlapping descriptions across tools (e.g. `analyze_content` vs `analyze_document`). Rename to remove semantic overlap.
- Built-in tools may be preferred over MCP tools. Counter this by emphasising unique data/context that built-ins cannot access.
- `tool_choice` modes: ``auto`` (default), ``any`` (must call SOME tool — guarantees structured output), ``{type:tool, name:X}`` (force a specific first step).
- JSON Schema with `tool_use` is the most reliable way to get structured output. It eliminates SYNTAX errors but not SEMANTIC errors.
- Schema design: required only when always present; nullable types for optional info; ``enum`` with ``other``+detail string and ``unclear`` for honest fallback.

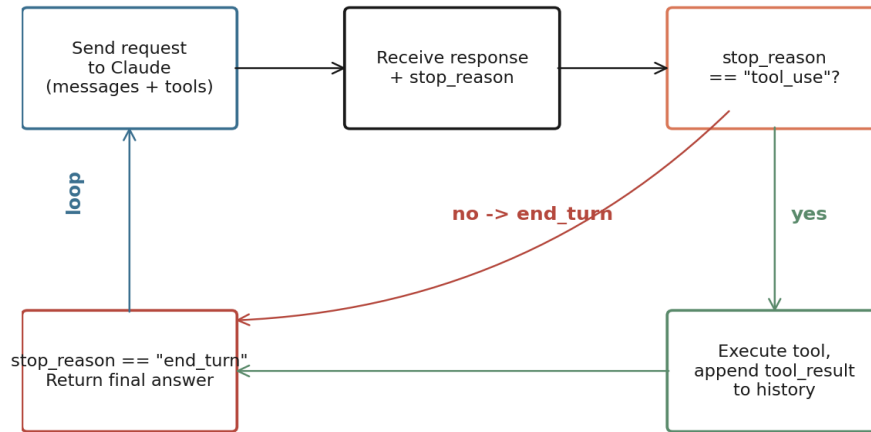
tool_choice — Three Modes



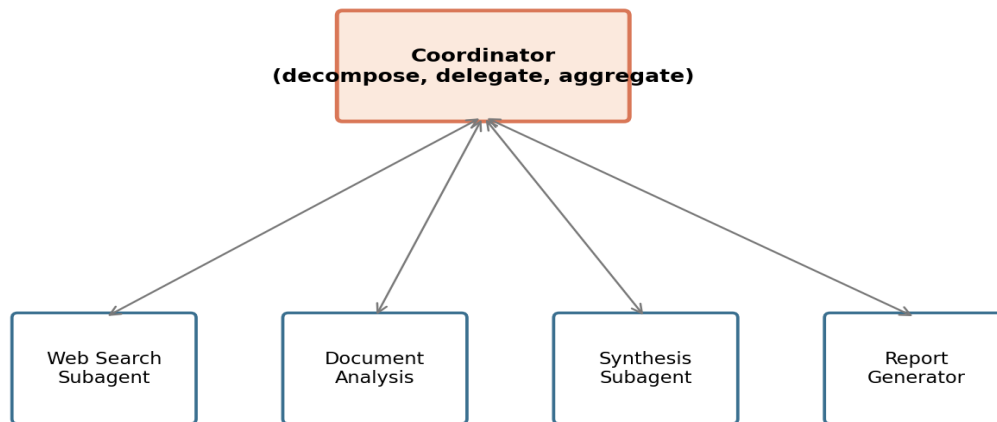
3. Claude Agent SDK — Building Agentic Systems

- Agentic loop: send → check `stop_reason` → if `tool_use`, run tool, append `tool_result`, loop → if `end_turn`, finish. The ONLY reliable completion signal is `stop_reason == 'end_turn'`.
- Anti-patterns: parsing assistant text for completion, using `max_iterations` as primary stop, treating presence of text content as 'done'.
- AgentDefinition fields: name, description, system_prompt, allowed_tools (least privilege).
- Hub-and-spoke: a coordinator owns decomposition, delegation, aggregation, error handling. All inter-agent comms route through the coordinator for observability.
- Subagents have ISOLATED context — they do NOT inherit the coordinator's history. Pass full required context explicitly in the Task prompt.
- Spawn subagents via the `Task` tool (must be in coordinator's `allowed_tools`). Multiple Task calls in one coordinator response run IN PARALLEL.
- Hooks intercept lifecycle events. `PostToolUse` normalises/trims tool results; `PreToolUse` blocks/redirects calls (e.g. block refunds > \$500).
- Hooks provide DETERMINISTIC guarantees; prompts give PROBABILISTIC compliance. Use hooks for financial / legal / safety constraints.
- Sessions: `--resume` continues prior context (risk: stale results). `fork_session` branches off a shared point for parallel approach exploration.

The Agentic Loop (model-driven)



Hub-and-Spoke Multi-Agent Architecture



Subagents have ISOLATED context — pass full context explicitly via Task prompt

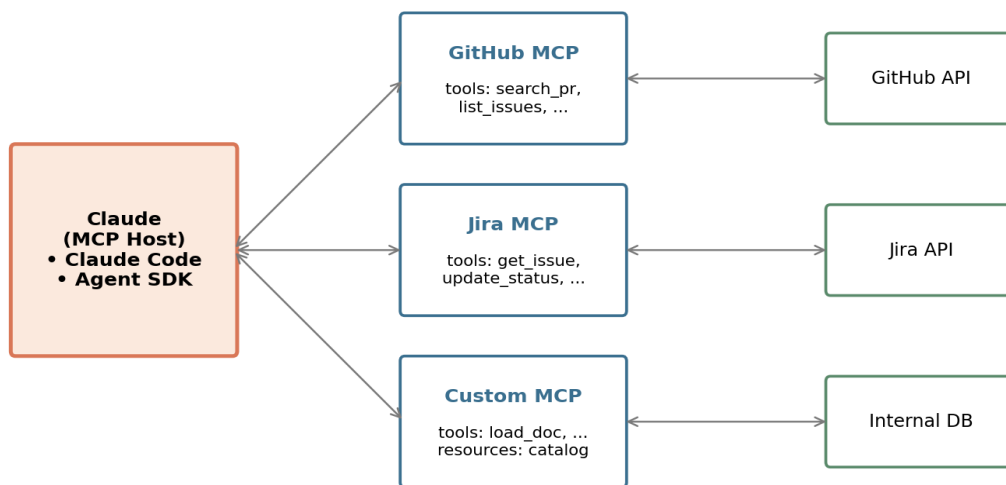
Hooks vs Prompt Instructions — When to Use Which

HOOKS (Code)	PROMPT INSTRUCTIONS
Deterministic — 100%	Probabilistic — >90%, never 100%
• PreToolUse: block / redirect tool calls • PostToolUse: normalize results, trim fields • Always enforced; not subject to drift • Best for: financial, legal, compliance, safety • Example: block refund > \$500	• "Always verify customer first" • "Try to resolve before escalating" • Fast to change; flexible • Best for: preferences, formatting, soft policy • Drifts as conversation grows

4. Model Context Protocol (MCP)

- Open protocol that exposes Tools (actions), Resources (read-only data), and Prompts (templates) to a Claude host.
- When a host connects to an MCP server, all tools are auto-discovered and merged with tools from other connected servers.
- Project config: `~/.mcp.json` in repo root (in VCS). Use ``${ENV_VAR}`` substitution so secrets are NOT committed.
- User config: `~/.claude.json` for personal/experimental servers, NOT shared via VCS.
- Prefer existing community MCP servers (GitHub, Jira, Slack); build custom only for unique team workflows.
- Errors: set ``isError: true`` and return STRUCTURED metadata: ``errorCategory`` (transient/validation/business/permission), ``isRetryable``, message, attempted_query, partial_results, alternative approaches. Generic 'Operation failed' breaks coordinator decision-making.
- Resources act as a 'map': agent reads catalogues / schemas / docs without exploratory tool calls.

Model Context Protocol (MCP) Architecture

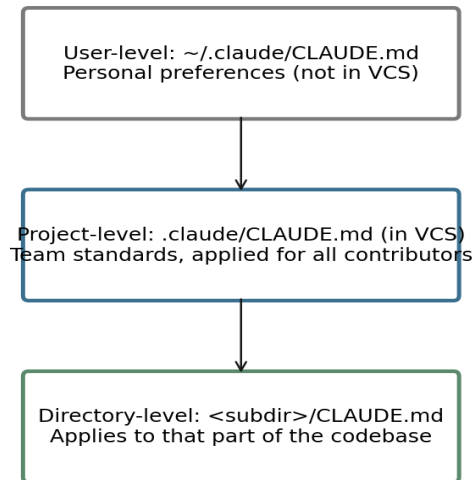


MCP exposes 3 primitives: Tools (actions), Resources (read-only data), Prompts (templates)

5. Claude Code — Configuration & Workflows

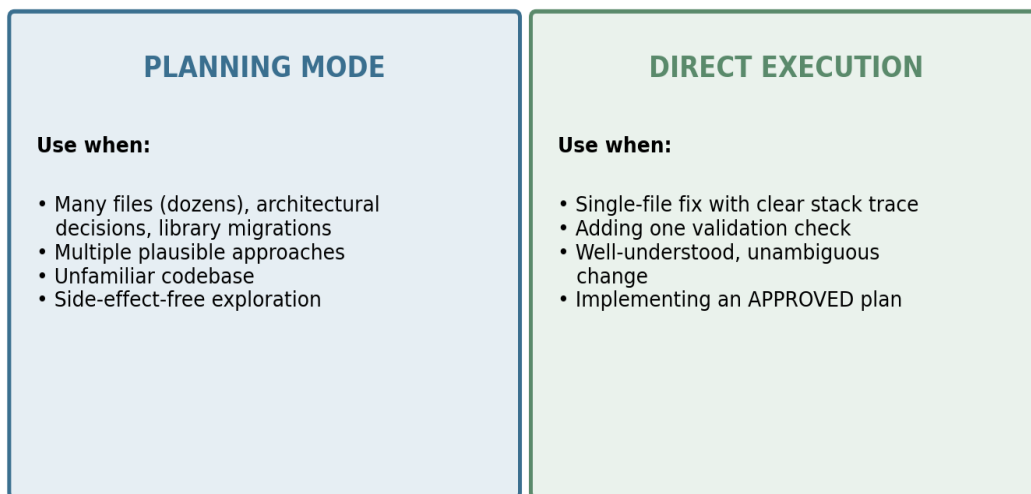
- CLAUDE.md hierarchy: User (~/.claude/CLAUDE.md) → Project (.claude/CLAUDE.md or root CLAUDE.md, in VCS) → Directory (subdir CLAUDE.md). Project-level guarantees teammates inherit standards.
- @path imports modularise CLAUDE.md (e.g. `@./standards/testing.md`). Max nesting depth is 5; relative paths resolve from the importing file.
- .claude/rules/ contains topical files with YAML frontmatter `paths` globs — loaded ONLY when editing matching files. Ideal for cross-cutting conventions like tests scattered across the codebase.
- Custom slash commands live in .claude/commands/ (project, in VCS) or ~/.claude/commands/ (personal).
- Skills (.claude/skills//SKILL.md) extend slash commands with frontmatter: `context: fork` (isolated subagent), `allowed-tools` (least privilege), `argument-hint` (prompts for arg).
- Personal skills at ~/.claude/skills//SKILL.md OVERRIDE same-named project skills for that user.
- Planning mode: investigate-only (Read/Grep/Glob), produces an approval-gated plan. Use for large changes, multiple approaches, architectural decisions, unfamiliar codebases.
- Direct execution: small, well-understood, single-file changes.
- Explore subagent: isolates verbose codebase exploration so it does not pollute main context.
- /compact: summarise prior history (risk: numeric/date precision loss). /memory: edit CLAUDE.md to persist facts across sessions.
- CI/CD: use `claude -p` (or --print) for non-interactive mode. Combine with --output-format json + --json-schema for parseable findings (e.g. inline PR comments via GitHub API).
- Use a SECOND independent Claude instance to review code generated by the first — avoids confirmation bias from generation context.
- When re-reviewing after new commits, include prior findings in context and instruct Claude to report only new/unresolved issues to avoid duplicate comments.

CLAUDE.md Hierarchy



All three levels merge into the active context. Use `@path` imports & `.claude/rules/` to modularize.

Planning Mode vs Direct Execution



6. Prompt Engineering — Advanced Techniques

- Few-shot (2–4 input/output examples) outperforms vague instructions for ambiguity, formatting, edge cases, informal measurements.
- Add normalisation rules (dates → ISO 8601, currency as numeric+code, percentages as decimal) to prevent semantic drift in structured output.
- Replace vague instructions with EXPLICIT criteria: what to flag vs ignore, severity definitions with code examples for each level.
- Prompt chaining: split tasks into sequential focused passes (e.g. per-file analysis then a cross-file integration pass) to avoid attention dilution.
- Use prompt chaining for predictable pipelines; use dynamic decomposition for open-ended investigations.
- Interview pattern: Claude asks clarifying questions before implementing in unfamiliar domains or when multiple viable approaches exist.
- Validation + retry-with-feedback: re-prompt with the original document, the prior wrong extraction, and the specific validation error. Pydantic is the canonical validation tool.
- Retry helps with format/structural/arithmetic issues; it does NOT help when the information is absent from the source.
- Self-correction: extract `stated\_total` AND `calculated\_total`; flag `conflict\_detected: true` for downstream handling.

Few-Shot Prompting — Why It Works

Vague instruction

"Be more conservative"

→ *Interpreted many ways. Inconsistent.*

Few-shot examples (2-4)

Q: "My order is broken"

A: get_customer → lookup_order → diagnose
(damaged item – need order details)

Q: "Get me a manager"

A: escalate_to_human (immediate, no further investigation)

→ *Unambiguous decision logic; model GENERALIZES the pattern to new cases.*

7. Message Batches API

- 50% cost saving vs synchronous; processing window up to 24h with NO latency SLA.
- Batch is asynchronous and 'fire-and-forget' — multi-turn tool calling within one request is NOT supported (a fundamental limit).
- `custom\_id` correlates each request with its response; on partial failure, resubmit only the failed custom_ids.
- Decision rule: blocking workflow (pre-merge, interactive review, agent loops) → synchronous. Scheduled workload (overnight reports, weekly audits, bulk extraction) → batch.
- SLA planning: if you need results in 30h and batches take up to 24h, your submission window is the remaining 6h.

Sync API vs Message Batches API

Workload	Choose	Why
Pre-merge PR check	Synchronous	Developer waits
Interactive code review	Synchronous	Real-time
Iterative tool-calling agent	Synchronous	Batch ≠ multi-turn
Overnight tech-debt report	Batch (50% off)	≤24h OK
Weekly security audit	Batch (50% off)	≤24h OK
10 000 invoices extraction	Batch (50% off)	Throughput

8. Task Decomposition Strategies

- Fixed pipeline (prompt chaining): every step pre-defined — extract metadata → extract data → validate → enrich → output.
- Dynamic adaptive decomposition: subtasks are generated from intermediate findings (legacy test coverage analysis, exploratory investigations).
- Multi-pass code review: per-file local analysis + a separate cross-file integration pass. Larger context windows do NOT fix attention dilution — focused passes do.
- Avoid 'consensus over multiple full reviews' — it suppresses real bugs by demanding agreement across inconsistent runs.

9. Escalation & Human-in-the-Loop

- Reliable triggers: explicit human request, policy gap (request not covered), inability to make progress, financial threshold (enforce via HOOK), multiple matches when identifying a customer.
- Unreliable triggers: sentiment analysis (mood $\neq$ complexity), self-rated confidence 1–10 (poorly calibrated — model can be confidently wrong), bespoke trained classifiers (over-engineering).
- Pattern: acknowledge emotion → propose concrete resolution → escalate only if customer reiterates the desire for a human (or explicitly asks first time).
- Structured handoff to human: customer_id, name, issue_summary, order_id, root_cause, actions_taken[], refund_amount, recommended_action, escalation_reason. Operators see ONLY this — make it self-contained.
- Confidence calibration: per-field scores; calibrate against labelled validation set; route low confidence to human review.
- Stratified random sampling: a 97% aggregate accuracy can hide 40% errors on a specific document type.

Escalation: Reliable vs Unreliable Triggers

RELIABLE

- ✓ Customer explicitly asks "get me a manager"
- ✓ Policy gap (request not covered by policy)
- ✓ Cannot make progress after reasonable attempts
- ✓ Financial op above threshold (enforce via HOOK)
- ✓ Multiple matches when searching customer

UNRELIABLE

- ✗ Sentiment analysis (mood $\neq$ complexity)
- ✗ Self-rated confidence 1–10 (poorly calibrated)
- ✗ Auto-classifier (overengineered, needs data)
- ✗ First sign of frustration (acknowledge, then resolve)

10. Error Handling in Multi-agent Systems

- Categories: Transient (retry w/ backoff), Validation (fix input & retry), Business (explain + alternative), Permission (escalate).
- Subagent does LOCAL recovery (1–2 retries) for transient failures; only propagates non-recoverable errors to coordinator.
- Anti-patterns: generic 'service unavailable' status (no recovery info), silent suppression (empty result == success), aborting whole workflow on one failure, infinite subagent retries.
- Distinguish ACCESS FAILURE (timeout — retry decision) from VALID EMPTY RESULT ('0 results' — successful query).
- Coverage annotations in synthesis: 'FULL COVERAGE' / 'PARTIAL COVERAGE — search timeout' to communicate uncertainty downstream.

MCP / Tool Error Categories

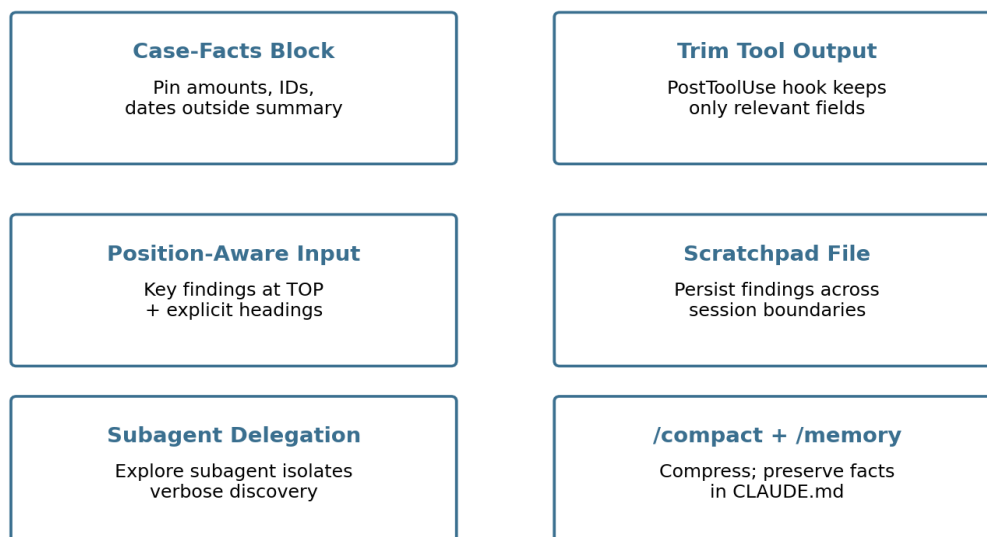
Category	Examples	Action
Transient	Timeout, 503, network	Retry w/ backoff
Validation	Bad input, missing field	Fix input & retry
Business	Policy violation, threshold	Explain to user; offer alternative
Permission	Access denied	Escalate to human

Always return *STRUCTURED* errors: *errorCategory*, *isRetryable*, *message*, *attempted_query*, *partial_results*.

11. Context Management in Production

- Extract a Case-Facts block (customer_id, order_id, amounts, dates, issue, request, status) and pin it OUTSIDE the summarised history.
- Trim verbose tool results via PostToolUse hook (40+ fields → keep 5).
- Position-aware input: KEY FINDINGS at the TOP, action items at the BOTTOM, detailed results in middle with explicit headings (mitigates lost-in-the-middle).
- Scratchpad files (e.g. investigation-scratchpad.md) survive context boundaries and new sessions.
- Delegate exploratory work to subagents (Explore subagent) — keeps main context clean.
- Constrained subagent budgets: minimal context, structured-only return, restricted allowed_tools.
- Crash recovery: each subagent persists state to agent-state/.json; coordinator reads manifest.json on resume.

Context-Window Management Strategies



12. Provenance Preservation in Multi-source Synthesis

- Always preserve claim → source mappings (URL, source name, publication date, methodology) — do not let attribution dissolve in summarisation.
- Conflicting data: keep both values with attribution; let the coordinator decide (do not arbitrarily pick one).
- Include dates so temporal differences are not misread as contradictions.
- Render by content type: tables for financial, prose for analysis, structured lists for technical findings, chronological for time series.

13. Claude Code Built-in Tools

- Glob — find files by name/extension pattern (`**/*.test.tsx`).
- Grep — search WITHIN file contents (function names, errors, imports).
- Read — full file load. Write — create new file. Edit — precise replace via unique string match.
- When Edit fails on non-unique match: fall back to Read → modify in memory → Write.
- Investigation strategy: incremental — Grep for entry points → Read those files → Grep for usages → Read consumers → repeat until full picture.

3. Domain-by-Domain Notes

Each domain is broken down into the specific knowledge and skills the official exam guide tests. Use these as your final-week checklists.

Domain 1 — Agent Architecture & Orchestration (27%)

Designing Agentic Loops

- Lifecycle: send → stop_reason check → tool_use vs end_turn → loop with appended tool_result.
- Model-driven, not hard-coded decision tree.
- Anti-patterns: text-parsing for completion, max_iterations as primary stop.

Multi-agent Orchestration

- Hub-and-spoke topology with coordinator-mediated communication for observability.
- Subagents have ISOLATED context — pass everything explicitly.
- Coordinator owns dynamic subagent selection, error handling, aggregation.
- Watch for OVERLY NARROW decomposition (assigning subtasks that miss whole subdomains).

Subagent Spawning & Context Passing

- Task tool spawns subagents (must be in coordinator's allowed_tools).
- Multiple Task calls in one turn run in parallel.
- Pass full required outputs from prior agents — never assume inheritance.
- Coordinator prompts: state goals + quality criteria, not step-by-step instructions.

Multi-step Workflows: Enforcement vs Guidance

- Programmatic preconditions (block lookup_order until get_customer returns a verified ID) > prompt instructions for critical sequences.
- Structured handoff payloads on escalation.

Agent SDK Hooks

- PostToolUse: normalise dates, trim noisy fields.
- PreToolUse: block policy-violating calls; redirect to escalation.
- Hooks deterministic; prompts probabilistic.

Task Decomposition

- Fixed pipelines (prompt chaining) vs dynamic adaptive decomposition.
- Per-file + integration passes for multi-file code review.

Sessions: Resume & Fork

- --resume continues; fork_session branches.
- Prefer fresh session with summary when results are stale.

Domain 2 — Tool Design & MCP Integration (18%)

Tool Interface Design

- Descriptions are the primary selection mechanism; include input formats, examples, edge cases, boundaries.
- Eliminate functional overlap by RENAMING (e.g. analyze_content → extract_web_results).
- Split general-purpose tools (fetch_url) into specialised ones (load_document with URL validation) — least privilege at the interface.

Structured Error Responses

- isError: true with errorCategory, isRetryable, message, attempted_query, partial_results, alternative_approaches.
- Distinguish access failures from valid empty results.
- Local recovery in subagents; propagate only what they can't resolve.

Tool Allocation & tool_choice

- ≤ ~5 well-scoped tools per agent; more than that hurts selection reliability.
- Subagent toolset matches its role.
- tool_choice 'any' guarantees structured output; force a specific tool to enforce a first step.

MCP Configuration

- Project .mcp.json (in VCS, \${ENV} for secrets) vs user ~/.claude.json (private).
- Tools from all connected servers are merged.
- Resources are read-only context maps.

Built-in Tools

- Grep / Glob / Read / Write / Edit / Bash purposes.
- Read+Write fallback when Edit can't find a unique match.

Domain 3 — Claude Code Configuration & Workflows (20%)

CLAUDE.md Hierarchy

- User → Project → Directory levels merge.
- Project-level (in VCS) is the only level new teammates inherit automatically.

- @path imports modularise; .claude/rules/ scopes via glob frontmatter.

Custom Slash Commands & Skills

- .claude/commands/ (legacy) and .claude/skills//SKILL.md (current) — both produce /name.
- Frontmatter: context: fork, allowed-tools, argument-hint.
- Personal ~/.claude/skills//SKILL.md overrides project skill for that user.

Path-specific Rules

- .claude/rules/.md with `paths: ['\*\*/\*.test.tsx']`.
- Loaded only when editing matching files — saves tokens.
- Better than dir-level CLAUDE.md when conventions span many directories (tests, migrations).

Planning Mode vs Direct Execution

- Plan first for large/architectural/multiple-approach changes.
- Direct for clear single-file fixes.
- Combine: plan, get approval, execute.

Iterative Refinement

- Concrete I/O examples > prose specs.
- Test-driven iteration; interview pattern for unknowns.

CI/CD Integration

- claude -p for non-interactive mode.
- --output-format json + --json-schema → parseable PR comments.
- Independent reviewer instance avoids confirmation bias.
- Include prior findings to suppress duplicate comments on follow-up commits.

Domain 4 — Prompt Engineering & Structured Output (20%)

Explicit Criteria

- Replace 'be conservative' with categorical rules and severity definitions tied to code examples.
- Disable categories with high false-positive rates to restore developer trust.

Few-shot Prompting

- 2–4 targeted examples with rationale for ambiguous decisions.
- Show output format AND distinguish acceptable vs problematic patterns.

Structured Output via tool_use + JSON Schema

- Eliminates SYNTAX errors; not semantic ones.
- Mark optional fields nullable so the model returns null rather than fabricating.
- Use enums with `other` + detail and `unclear` to capture out-of-distribution cases.
- tool_choice 'any' for guaranteed call; force specific tool to lock the first step.

Validation, Retries, Feedback

- Retry-with-error-feedback: original doc + wrong extraction + concrete error message.
- Pydantic semantic validators for business rules (sum(items) == total, start < end).
- Self-correction: stated_total + calculated_total + conflict_detected.

Batch Processing Strategies

- Sync vs Batch decision = blocking vs scheduled.
- Iterate prompts on a sample BEFORE bulk processing.
- Resubmit failed custom_ids only.

Multi-instance / Multi-pass Review

- Independent reviewer Claude instance.
- Per-file + integration pass for multi-file PRs.

Domain 5 — Context Management & Reliability (15%)

Conversation Context

- Send full history every request — model is stateless.
- Lost-in-the-middle: critical info goes at top or bottom.
- Trim verbose tool outputs via PostToolUse hook.
- Pin transactional facts in a Case-Facts block outside summarisation.

Escalation & Ambiguity

- Reliable triggers; ask for additional identifier on multiple matches; immediate handoff on explicit human request.

Error Propagation

- Structured error context, partial results, alternative approaches.
- Distinguish access failure from empty result.
- Coverage annotations in synthesis output.

Large-codebase Context Efficiency

- Explore subagent isolates discovery output.

- Scratchpad files preserve findings across sessions.
- /compact when verbose; /memory persists across sessions via CLAUDE.md.

Human Oversight & Confidence Calibration

- Field-level confidence scores; calibrate on labelled set.
- Stratified random sampling — analyse accuracy per document type / field.

Provenance Preservation

- claim → source mappings; preserve conflicting values with attribution.
- Always include publication dates.

4. Exam Scenarios

Scenarios are described by the official exam guide. Internalise each as a complete system: what tools it uses, where it usually breaks, which trade-offs the exam will probe.

Customer Support Agent

Returns, billing disputes, account issues. Tools: `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`. Target: 80%+ first-contact resolution. Common test points: tool selection on ambiguous wording, escalation calibration, programmatic preconditions, `PostToolUse` normalisation, Case-Facts blocks.

Code Generation with Claude Code

Generation, refactoring, debugging, documentation. Test points: `CLAUDE.md` hierarchy, `.claude/rules/`, project vs personal skills with context: fork, planning mode vs direct execution, MCP server configuration via `.mcp.json`.

Multi-agent Research System

Coordinator + web-search + document analysis + synthesis + report generator. Test points: hub-and-spoke topology, parallel Task spawn, explicit context passing, structured error propagation, conflicting data with attribution, coverage annotations, lost-in-the-middle mitigations, narrow-decomposition root cause.

Developer Productivity Tools

Codebase exploration, boilerplate generation, automated routine tasks. Built-in tools (`Read/Write/Bash/Grep/Glob`) + MCP integrations. Test points: tool-selection reliability, Explore subagent, scratchpad files, incremental investigation strategy.

Claude Code for Continuous Integration

`claude -p` in CI for code review and test generation. Test points: `--output-format json` + `--json-schema` for parseable findings, independent reviewer instance, prior-findings context to avoid duplicate comments, sync vs Batch decision, attention dilution on multi-file PRs.

Structured Data Extraction

Unstructured documents → schema-validated output. Test points: `tool_use` + JSON schema, nullable & enum-with-other fields, validation+retry loops, calibration, stratified sampling, batch processing with `custom_id`.

Conversational AI Architecture Patterns

Multi-turn chat over many sessions. Test points: context window management, prompt drift, instruction reinforcement, prefill, Case-Facts blocks, semantic retrieval for very long histories, preview/execute token-binding for safe destructive tools.

Agentic AI Tools

(Less documented in early exam writeups.) Generally reuses the same themes: tool design, sequencing enforcement, hooks, error envelopes, structured outputs, and reliability.

5. Quick-Reference Cheat Sheet

Print this and revisit the night before the exam. Each block is one exam-relevant idea distilled to its smallest useful form.

Stop reasons

- `end_turn` → finished (only reliable completion signal)
- `tool_use` → run tool, append `tool_result`, loop
- `max_tokens` → truncated; raise limit
- `stop_sequence` → application-specific

tool_choice

- "auto" → model decides text vs tool (default)
- "any" → must call SOME tool (guarantees structured output)
- `{"type": "tool", "name": "X"}` → must call X (force first step)

Hooks vs Prompts

- Hooks: deterministic 100% — financial, legal, safety, compliance
- Prompts: probabilistic — preferences, formatting, soft policy

Subagents (Task tool)

- Coordinator's `allowed_tools` must include 'Task'
- ISOLATED context — pass everything explicitly
- Multiple Task calls in ONE turn run in PARALLEL
- Return STRUCTURED data, not raw dumps

MCP error envelope

- `isError`: true
- `errorCategory`: transient | validation | business | permission
- `isRetryable`: bool
- `attempted_query`, `partial_results`, `alternative_approaches`

CLAUDE.md scope

- User: `~/claude/CLAUDE.md` (private)
- Project: `.claude/CLAUDE.md` or root `CLAUDE.md` (in VCS — only level teammates inherit)
- Directory: `/CLAUDE.md` (scoped to that area)
- Modularise via `@path` or `.claude/rules/.md`

.claude/rules/ frontmatter

- paths: ['src/api/**/*.ts'] — load only when editing matching files
- Use for cross-cutting conventions (tests scattered across repo)

Skill frontmatter

- context: fork → isolated subagent (avoid main-context pollution)
- allowed-tools: ['Read','Grep'] → least privilege
- argument-hint: 'Path to analyse'

CI/CD CLI

- claude -p (or --print) — non-interactive
- --output-format json + --json-schema — parseable findings
- Run a SECOND independent instance to review (avoids confirmation bias)
- Include prior findings to avoid duplicate comments on follow-up commits

Sync vs Batch

- Sync: blocking workflows, agent loops with multi-turn tool use, real-time UX
- Batch (50% off, ≤24h): overnight reports, weekly audits, bulk extraction
- Batch does NOT support multi-turn tool calling within one request

Escalation triggers (reliable)

- Explicit human request
- Policy gap / silent on request
- Cannot make progress
- Financial threshold (enforce via hook)
- Multiple customer matches → ask for additional identifier

Anti-patterns to recognise

- Parsing assistant text for completion (use stop_reason)
- max_iterations as primary stop
- Generic 'operation failed' error responses
- Aborting whole workflow on one subagent failure
- Sentiment analysis for escalation
- Self-rated confidence 1–10 for routing
- Single-pass review of 14 files (attention dilution)
- Treating empty result == access failure

MCP / Tool Error Categories

Category	Examples	Action
Transient	Timeout, 503, network	Retry w/ backoff
Validation	Bad input, missing field	Fix input & retry
Business	Policy violation, threshold	Explain to user; offer alternative
Permission	Access denied	Escalate to human

Always return **STRUCTURED** errors: `errorCategory`, `isRetryable`, `message`, `attempted_query`, `partial_results`.

Hooks vs Prompt Instructions — When to Use Which

HOOKS (Code)

Deterministic — 100%

- PreToolUse: block / redirect tool calls
- PostToolUse: normalize results, trim fields
- Always enforced; not subject to drift
- Best for: financial, legal, compliance, safety
- Example: block refund > \$500

PROMPT INSTRUCTIONS

Probabilistic — >90%, never 100%

- "Always verify customer first"
- "Try to resolve before escalating"
- Fast to change; flexible
- Best for: preferences, formatting, soft policy
- Drifts as conversation grows

Sync API vs Message Batches API

Workload	Choose	Why
Pre-merge PR check	Synchronous	Developer waits
Interactive code review	Synchronous	Real-time
Iterative tool-calling agent	Synchronous	Batch ≠ multi-turn
Overnight tech-debt report	Batch (50% off)	≤24h OK
Weekly security audit	Batch (50% off)	≤24h OK
10 000 invoices extraction	Batch (50% off)	Throughput

6. Practice Questions with Explanations

These 20 questions cover the most common exam patterns across all 5 domains. Every answer below is the official correct option with the reason it beats the seductive alternatives.

Q1. (Customer Support Agent) Logs show that in 12% of cases the agent skips `get_customer` and calls `lookup_order` using only the customer-provided name, leading to incorrect refunds. Which change is MOST effective?

- A. Add a programmatic precondition that blocks `lookup_order` and `process_refund` until `get_customer` returns a verified ID
- B. Strengthen the system prompt requiring verification first
- C. Add few-shot examples of correct sequencing
- D. Add a routing classifier

Answer: A

Why: Critical sequencing must be deterministic. Hooks/preconditions guarantee 100% compliance; prompts (B,C) are probabilistic. D solves a different problem (availability, not ordering).

Q2. (Customer Support Agent) Agent often calls `get_customer` instead of `lookup_order` for order questions; tool descriptions are minimal and similar. First step?

- A. Add few-shot examples
- B. Expand each tool description with input formats, examples, edge cases, boundaries
- C. Add a routing layer
- D. Merge the tools

Answer: B

Why: Descriptions are the model's primary selection mechanism — fix the root cause first. Lowest-effort, highest-impact.

Q3. (Customer Support Agent) Agent resolves only 55% of issues vs 80% target; escalates simple cases and tries to handle complex policy exceptions. How to improve calibration?

- A. Explicit escalation criteria with few-shot examples
- B. Self-rated confidence 1–10 with auto-escalation
- C. Train a separate classifier
- D. Sentiment analysis

Answer: A

Why: Decision-boundary problem → explicit criteria + examples. Self-rated confidence is poorly calibrated; sentiment ≠ complexity; classifier is over-engineering.

Q4. (Code Generation with Claude Code) You need a /review slash command available to the whole team after cloning the repo. Where do you put it?

- A. `.claude/commands/` in the project repo
- B. `~/claude/commands/`
- C. Root `CLAUDE.md`
- D. `.claude/config.json`

Answer: A

Why: Project-level commands in VCS are inherited by every clone. `~/` is personal-only.

Q5. (Code Generation with Claude Code) You need to restructure a monolith into microservices (dozens of files, boundary decisions). Approach?

- A. Planning mode: explore → understand dependencies → design
- B. Direct execution incrementally
- C. Direct execution with detailed up-front instructions
- D. Direct execution; switch to planning when it gets hard

Answer: A

Why: Planning mode is designed for large, multi-approach, architectural changes. B/C/D risk expensive rework.

Q6. (Code Generation with Claude Code) Codebase has different conventions per area; tests are co-located with code. You want conventions auto-applied. Best approach?

- A. `.claude/rules/` files with YAML frontmatter ``paths`` glob patterns
- B. Put everything in root `CLAUDE.md`
- C. Skills in `.claude/skills/`
- D. `CLAUDE.md` in every directory

Answer: A

Why: paths-scoped rules load only when editing matching files — ideal for cross-cutting conventions like tests scattered across the codebase.

Q7. (Multi-agent Research System) Reports cover only visual art; coordinator decomposed into 'AI in digital art', 'AI in graphic design', 'AI in photography'. Likely root cause?

- A. Synthesis agent doesn't detect gaps
- B. Coordinator decomposed too narrowly
- C. Web-search agent isn't thorough
- D. Document analysis filters out non-visual sources

Answer: B

Why: Subagents executed correctly — the issue is what they were assigned. Watch for coordinator decomposition that misses whole subdomains.

Q8. (Multi-agent Research System) A web-search subagent times out. Best error propagation to coordinator?

- A. Structured error: `failure_type`, `query`, `partial_results`, `alternative_approaches`
- B. Internal retries, then return generic 'search unavailable'
- C. Catch the timeout and return empty result marked successful
- D. Propagate timeout exception, terminate workflow

Answer: A

Why: Coordinator needs context to choose: `retry/alternative/partial/skip`. B hides info; C masks failure; D over-aborts.

Q9. (Multi-agent Research System) Document agent hits 2 contradictory credible sources for a key statistic. Best handling?

- A. Apply credibility heuristics, footnote the discrepancy
- B. Include both, unmarked, let synthesis decide
- C. Stop and ask coordinator which is authoritative
- D. Complete with both values, explicitly annotate the conflict with attribution, let coordinator reconcile

Answer: D

Why: Preserve both values + attribution; reconciliation is the coordinator's job (broader context).

Q10. (Multi-agent Research System) fetch_url on the document analysis agent is being misused for ad-hoc web search. Most effective fix?

- A. Replace fetch_url with load_document that validates URLs are document formats
- B. Remove fetch_url, route all URL fetches via coordinator/web-search agent
- C. Block known search-engine domains
- D. Add prompt instructions saying 'don't use for search'

Answer: A

Why: Constrain capability at the interface — least privilege. Makes misuse architecturally impossible, not merely discouraged.

Q11. (Claude Code for CI) Pipeline runs `claude "Analyze this PR for security issues"` and hangs. Fix?

- A. Use --batch flag
- B. Use -p (or --print) flag
- C. Redirect stdin from /dev/null
- D. Set CLAUDE_HEADLESS=true

Answer: B

Why: -p / --print is the documented non-interactive mode for CI/CD.

Q12. (Claude Code for CI) Manager wants to move BOTH (1) blocking pre-merge checks and (2) overnight tech-debt reports to Batch API for 50% savings. How to evaluate?

- A. Move both to batch with polling
- B. Use sync for pre-merge checks; batch for overnight reports
- C. Keep both sync to avoid ordering issues
- D. Move both to batch with sync fallback

Answer: B

Why: Batch up to 24h is unsuitable for blocking workflows where developers wait, but ideal for overnight scheduled work.

Q13. (Claude Code for CI) Iterative review uses tool calls to fetch related files. Why is Batch API fundamentally unsuitable here?

- A. Batch lacks correlation IDs
- B. Async fire-and-forget cannot intercept tool calls mid-request
- C. Batch doesn't support tools in request params
- D. 24h latency is too slow

Answer: B

Why: Batch is single request → single response. Multi-turn tool calling within one logical interaction needs synchronous round trips.

Q14. (Claude Code for CI) Findings are accurate but not actionable ('complex routing logic', 'potential null pointer'). Adding instructions doesn't help. Most reliable fix?

- A. More explicit instructions
- B. Larger context window
- C. Two-pass: identify, then generate fixes
- D. 3–4 few-shot examples showing the exact required format (issue, location, concrete fix)

Answer: D

Why: Instruction-only changes produce inconsistency; concrete examples lock in format reliably.

Q15. (Structured Data Extraction) After two iterations, output structure still mismatches expectations (nesting, timestamp format). Best next step?

- A. JSON schema validation after each iteration
- B. Provide 2–3 concrete input/output examples of the expected transformation
- C. Rewrite requirements with more technical precision
- D. Ask Claude to explain its current understanding

Answer: B

Why: Examples remove prose ambiguity directly. Schema validation enforces structure but not the desired mapping.

Q16. (Conversational AI Architecture Patterns) Two messages after the user said 'I love jazz', Claude asks 'What genres do you enjoy?' Most likely cause?

- A. Need a vector DB for memory
- B. Context window exceeded
- C. Claude API needs session_id
- D. Application isn't including prior messages in the messages array

Answer: D

Why: Claude is stateless. Every request must include the full prior history.

Q17. (Conversational AI Architecture Patterns) remove_team_member has dry_run; agent bypasses preview by calling dry_run=false directly. Most reliable enforcement?

- A. Server-side validation requiring a preceding dry_run within 60s
- B. Mark the tool as 'requires confirmation' in orchestration
- C. Detailed instructions + few-shot in tool description
- D. Two tools: preview_remove_member returns a single-use confirmation token; execute_remove_member requires that token

Answer: D

Why: Token-binding makes execution architecturally impossible without a preview — code-level enforcement, not LLM compliance.

Q18. (Conversational AI Architecture Patterns) Production: customers reference '15% discount mentioned earlier' but the agent cites wrong values; details were 20+ turns ago and got summarised vaguely. Best fix?

- A. Raise summarisation threshold to 85%
- B. Store full history externally and retrieve on 'as I mentioned'
- C. Extract transactional facts (amounts, dates, IDs) into a persistent Case-Facts block included every prompt
- D. Revise summarisation prompt to preserve numbers verbatim

Answer: C

Why: Summarisation inevitably loses precision. Pin facts OUTSIDE the summarisable history so they're always present.

Q19. (Customer Support Agent) Tools return Unix timestamps, ISO dates, numeric status codes; some tools are 3rd-party MCP servers you cannot modify. Most maintainable normalisation?

- A. PostToolUse hook intercepts outputs and normalises before the agent sees them
- B. Modify tools you control + wrap 3rd-party
- C. A normalize_data tool the agent calls after each retrieval
- D. Document formats in the system prompt

Answer: A

Why: Centralised, deterministic, code-level transformation that works uniformly regardless of source.

Q20. (Customer Support Agent) Agent loop: when do you stop calling Claude vs continue with another tool result?

- A. Check stop_reason: continue on tool_use, stop on end_turn
- B. Parse text for 'I'm done' / 'anything else?'
- C. Stop after max iterations
- D. Stop if assistant produced any text content

Answer: A

Why: stop_reason is the explicit, reliable signal. Everything else is heuristic and breaks.

7. Practical Exercises (Hands-on Labs)

Reading isn't enough — the exam asks you to choose between implementations. The fastest way to internalise the material is to BUILD the four reference systems below, in order.

Exercise 1 — Multi-tool Agent with Escalation

Domains: 1, 2, 5

- Define 3–4 MCP tools with rich descriptions (include 2 similar to test selection).
- Implement an agent loop checking `stop_reason` (`tool_use` vs `end_turn`).
- Add structured error responses (`errorCategory`, `isRetryable`, `message`).
- Add a `PreToolUse` hook blocking refunds > \$500 and routing to escalation.
- Test with a multi-aspect customer request (refund + shipping update).

Exercise 2 — Claude Code for Team Development

Domains: 3, 2

- Project-level `CLAUDE.md` with universal standards.
- `.claude/rules/` files with paths globs (`api`, `tests`, `deployments`).
- A skill at `.claude/skills//SKILL.md` with context: `fork` and `allowed-tools`.
- `.mcp.json` with `${ENV}` substitution for a community server (GitHub).
- Test planning mode vs direct execution on tasks of different sizes.

Exercise 3 — Structured Data Extraction Pipeline

Domains: 4, 5

- Define an extraction tool with JSON schema (`required/nullable/enum` with `'other'`).
- Validation loop: on error, retry with original document + wrong extraction + concrete error.
- Few-shot examples for inline citations vs bibliography references vs informal measurements.
- Batch run via Message Batches API on 100 docs; resubmit failed `custom_ids` only.
- Field-level confidence; route low-confidence and ambiguous-source docs to human review.

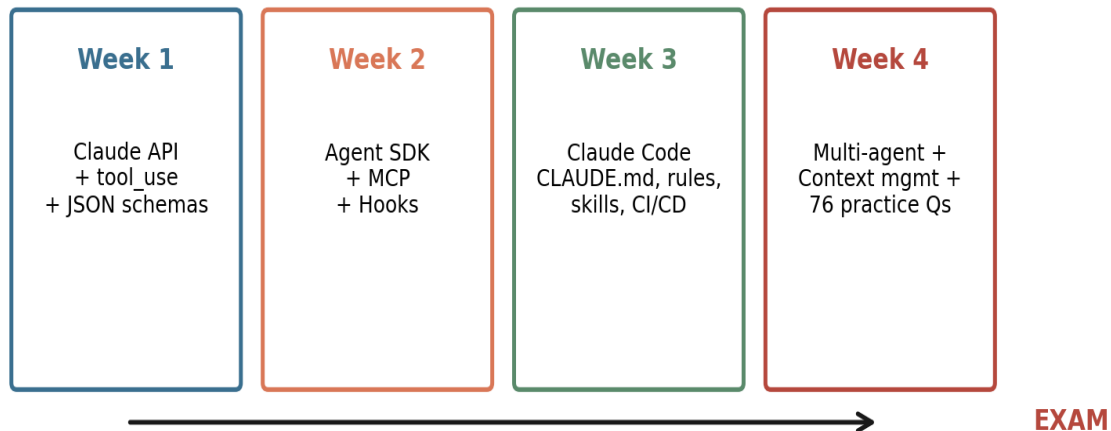
Exercise 4 — Multi-agent Research Pipeline

Domains: 1, 2, 5

- Coordinator with `allowed_tools=['Task', ...]`; 2+ subagents (web search + doc analysis).
- Spawn subagents in parallel via multiple Task calls in one coordinator turn.
- Require structured subagent output: claim, quote, source URL, publication date.
- Simulate a subagent timeout — return structured error context, continue with partial results, annotate coverage in synthesis.
- Test with conflicting data — preserve both values with attribution.

8. Study Roadmap & Resources

Suggested 4-Week Study Roadmap



Official documentation to study

- Claude API — Messages: platform.claude.com/docs/en/api/messages
- Claude API — Tool Use: platform.claude.com/docs/en/build-with-claude/tool-use
- Claude API — Message Batches: platform.claude.com/docs/en/build-with-claude/message-batches
- Agent SDK — Overview / Hooks / Subagents / Sessions: [platform.claude.com/docs/en/agent-sdk/...](https://platform.claude.com/docs/en/agent-sdk/)
- Model Context Protocol: modelcontextprotocol.io (Tools, Resources, Servers)
- Claude Code — Memory / Skills / Hooks / Sub-agents / MCP / GitHub Actions / Headless: [code.claude.com/docs/en/...](https://code.claude.com/docs/en/)
- Prompt Engineering Guide: platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
- Anthropic Cookbook (code examples): github.com/anthropics/anthropic-cookbook

Skilljar prep tracks (free)

- AI Fluency: Framework & Foundations
- Introduction to Agent Skills
- Building with the Claude API
- Claude Code in Action

- Intro to Model Context Protocol
- MCP: Advanced Topics
- Prompt Engineering Interactive Tutorial
- Real-world Prompting
- Tool Use with Claude
- Customer Service Agent (cookbook walkthrough)
- Multi-agent Research Pipelines
- Structured Data Extraction Workshop
- Claude Code in CI/CD

Community & Video

- YouTube prep playlist (PLFz7SvAnfqLpjPCPJBkUy077BC5ecuE8c) — exam walkthroughs and demos.
- github.com/paullarionov/claude-certified-architect — community study guide & practice questions.
- claudecertifications.com — free study guide and practice questions.
- FlashGenius / Tutorialsdodo / DEV.to / Towards AI — domain primers and worked questions.

9. Out-of-Scope Topics

Don't waste time on these — they will not appear on the exam. Knowing what NOT to study is itself a study tactic.

- Fine-tuning / training custom models
- API authentication, billing, account management
- Language/framework-specific implementation details (beyond schema/tool config)
- Hosting / infrastructure for MCP servers
- Claude internals (training, weights, RLHF, Constitutional AI)
- Embeddings & vector DB implementation details
- Computer use / browser automation
- Vision / image analysis
- Streaming API / SSE
- Rate limiting & quotas / detailed cost calculations
- OAuth, key rotation, auth protocol details
- Cloud-provider specific configurations (AWS / GCP / Azure)
- Performance benchmarks / model comparisons
- Prompt caching implementation details (beyond knowing it exists)
- Tokenisation specifics

10. Glossary

Term	Definition
Agentic loop	Send → check stop_reason → run tool on tool_use → loop. The model decides the next step.
AgentDefinition	Agent SDK config object: name, description, system_prompt, allowed_tools.
CLAUDE.md	Persistent instruction file for Claude Code. Loaded at user/project/directory levels.
.claude/rules/	Topical rule files with YAML frontmatter (paths globs) for conditional loading.
Coordinator	The hub agent that decomposes work, delegates to subagents, aggregates results.
custom_id	Per-request identifier in the Message Batches API for correlating responses & retries.
Edit tool	Claude Code tool that replaces a unique substring in a file.
end_turn	stop_reason value indicating Claude has finished its response.
fork_session	Clone a session at a shared point so two branches diverge independently.
Hook	Code that runs at a lifecycle event (PreToolUse / PostToolUse) — deterministic enforcement.
isError	MCP response flag indicating a tool failure; pair with structured error metadata.
JSON Schema	Structured-output contract used with tool_use; eliminates syntax errors.
Lost-in-the-middle	LLM tendency to under-attend to information in the middle of long inputs.
MCP	Model Context Protocol — open protocol exposing tools, resources, and prompts to a host.
Planning mode	Claude Code mode that investigates without making changes; produces an approved plan.
PostToolUse hook	Intercepts tool results before the model consumes them — normalisation, trimming.

PreToolUse hook	Intercepts outgoing tool calls — block, redirect, enforce policy.
Resource (MCP)	Read-only data exposed by an MCP server (catalogues, schemas, docs).
SKILL.md	Skill definition file under <code>.claude/skills//</code> with frontmatter (context: fork, allowed-tools).
stop_reason	Field on Claude responses driving the agentic loop: <code>end_turn</code> , <code>tool_use</code> , <code>max_tokens</code> , <code>stop_sequence</code> .
Subagent	Specialised agent spawned by the coordinator via the Task tool. Isolated context.
Task tool	Built-in tool that spawns a subagent in the Agent SDK / Claude Code.
tool_choice	<code>auto</code> <code>any</code> <code>{type:tool,name:X}</code> — controls forced tool calling.
tool_use	API mechanism for Claude to request tool calls in a structured form.